

Project Proposal:

Quantization for Efficient LLM Inference

1 Context and Motivation

Quantization is widely used because many LLMs are too expensive to run in full precision, especially on limited GPU memory. A lower-precision model can fit on cheaper hardware, allow larger batch sizes, or support longer contexts. However, quantization is not automatically better. A 4-bit model may use much less memory, but it may also lose accuracy, become slower in some runtimes, or fail more often on tasks that require reasoning or precise instruction following.

The literature provides several practical post-training methods. LLM.int8() [Dettmers et al., 2022](#) enables 8-bit weight loading with mixed-precision outlier handling. GPTQ [Frantar et al., 2022](#) and AWQ [Lin et al., 2023](#) use small calibration sets to push weights down to 4 bits with limited quality loss. SmoothQuant [Xiao et al., 2022](#) additionally targets activations to enable W8A8 inference. For 4-bit storage, the NF4 data type introduced together with QLoRA [Dettmers et al., 2023](#) is the standard choice in the `bitsandbytes` ecosystem and has been shown to recover most quality through LoRA [Hu et al., 2021](#) adapters. More recent work targets the KV cache itself, e.g. KIVI [Liu et al., 2024](#).

This project focuses on the practical trade-off these methods introduce. We will compare a full-precision baseline with several post-training quantization methods and measure both model quality and serving behavior. The project is valid even if the most aggressive method performs worse than the baseline, as long as we can clearly show where the degradation appears and whether the efficiency gains justify it.

2 Research Questions

1. Which quantization method gives the best quality–memory–speed trade-off for a 7B-class instruction-tuned LLM?
2. Are some capabilities more sensitive to quantization than others (reasoning, coding, QA, instruction following)?
3. Does quantization improve real inference speed, or does it mainly reduce memory usage?
4. *Optional:* can light QLoRA-style adaptation recover part of the quality lost after 4-bit quantization?

3 Core Experimental Setup

3.1 Main model

The core experiments use one 7B-class instruction-tuned model: **Qwen2.5-7B-Instruct** [Qwen Team, 2024](#). This keeps the project manageable and reduces uncontrolled variables. If time allows, we may repeat a subset of experiments on **Mistral-7B-Instruct** [Jiang et al., 2023](#) to check whether the conclusions generalize.

3.2 Quantization variants

We compare the following quantization methods. For calibration-based methods (GPTQ, AWQ), we will use the same small calibration set across methods that require calibration. This should make the comparison fairer.

4 Evaluation Plan

4.1 Quality metrics

We evaluate each model variant on a small set of benchmarks covering different capabilities. The exact list can be adjusted depending on runtime.

The core set is MMLU, GSM8K, HumanEval, and IFEval. More will be added if time allows.

Variant	Description	Priority
FP16 / BF16 baseline	Full/half-precision reference model	Core
8-bit	LLM.int8() Dettmers et al., 2022 or equivalent 8-bit loading	Core
4-bit NF4	bitsandbytes 4-bit loading with NF4 Dettmers et al., 2023	Core
4-bit GPTQ	Post-training quantization with calibration Frantar et al., 2022	Core
4-bit AWQ	Activation-aware post-training quantization Lin et al., 2023	Core
W8A8 / SmoothQuant	Quantized weights and activations Xiao et al., 2022	Optional
Quantized KV cache	Lower-precision KV cache for long-context inference Liu et al., 2024	Optional
QLoRA adaptation	LoRA Hu et al., 2021 adapters trained on top of a 4-bit model	Optional

Capability	Dataset / benchmark	Metric
General knowledge / QA	MMLU Hendrycks et al., 2020	Accuracy
Mathematical reasoning	GSM8K Cobbe et al., 2021	Accuracy / exact match
Code generation	HumanEval Chen et al., 2021	pass@1
Instruction following	IFEval Zhou et al., 2023	Prompt-level acc.
Judge reliability (<i>opt.</i>)	JudgeBench Tan et al., 2024	Pairwise judge accuracy

4.2 Efficiency metrics

For each model variant, we measure:

- model size on disk;
- peak GPU memory during loading;
- peak GPU memory during generation;
- prefill throughput;
- decode throughput;
- end-to-end latency;
- maximum batch size before OOM;
- behavior under short, medium, and long prompts.

These measurements matter because lower memory usage does not always imply faster inference. In some settings, quantization only helps fit the model into memory. In others, it improves throughput by reducing memory traffic or allowing larger batches.

5 Expected Results

We expect 8-bit quantization to stay close to the full-precision baseline in quality while reducing memory usage. We expect 4-bit methods to save more memory, but with a higher risk of quality loss. GPTQ and AWQ may perform better than simple 4-bit loading because they use calibration, but this has to be tested under the same setup. The final recommendation will depend on the full trade-off between quality, memory, and speed.

6 Minimal Scope and Extensions

Minimal successful version:

- one model: Qwen2.5-7B-Instruct [Qwen Team, 2024](#);
- five variants: baseline, 8-bit, 4-bit NF4, GPTQ, AWQ;
- four benchmarks: MMLU [Hendrycks et al., 2020](#), GSM8K [Cobbe et al., 2021](#), HumanEval [Chen et al., 2021](#), IFEval [Zhou et al., 2023](#);
- memory and speed measurements for each variant.

Extensions:

- add Mistral-7B-Instruct [Jiang et al., 2023](#) as a second model;
- add JudgeBench [Tan et al., 2024](#);
- add W8A8 / SmoothQuant [Xiao et al., 2022](#);
- add quantized KV cache [Liu et al., 2024](#);
- add QLoRA-style adaptation [Dettners et al., 2023](#).